

5. ОБРАБОТКА ОШИБОК (ERROR HANDLING)

Во всех программах, кроме самых простых, у вас будут методы, которые могут завершиться неудачей. В этой главе мы рассмотрим различные способы представления, обработки и распространения таких сбоев, а также преимущества и недостатки каждого из них. Мы начнём с изучения разных способов представления ошибок, включая перечисление (enumeration) и стирание (erasure), а затем разберём некоторые особые случаи ошибок, которые требуют иного приёма представления. Далее мы рассмотрим различные способы обработки ошибок и будущее обработки ошибок.

Стоит отметить, что лучшие практики обработки ошибок (error handling) в Rust до сих пор являются активной темой обсуждения, и на момент написания этой книги экосистема ещё не сошлась на едином, унифицированном подходе. Поэтому в этой главе мы сосредоточимся на основополагающих принципах и приёмах, а не на рекомендации конкретных крейтов или паттернов.

Представление ошибок (Representing Errors)

Когда вы пишете код, который может завершиться неудачей, самый важный вопрос, который стоит себе задать, — как ваши пользователи будут взаимодействовать с любыми возвращаемыми ошибками. Нужно ли пользователям точно знать, какая именно ошибка произошла, и подробности о том, что пошло не так, или же они просто запишут в журнал факт ошибки и продолжат работу, как смогут? Чтобы это понять, нам нужно рассмотреть, влияет ли природа ошибки на то, что вызывающий код делает при её получении. Это, в свою очередь, продиктует, как мы будем представлять разные ошибки.

У вас есть два основных варианта представления ошибок: перечисление (enumeration) и стирание (erasure). То есть вы можете либо *перечислить* в своём типе ошибки возможные состояния ошибки, чтобы вызывающий код мог их различать, либо просто предоставить вызывающему коду единственную *непрозрачную* (opaque) ошибку. Давайте обсудим оба этих варианта по

очереди.

Перечисление (Enumeration)

В нашем примере мы воспользуемся библиотечной функцией, которая копирует байты из некоторого входного потока в некоторый выходной поток, во многом подобно `std::io::copy`. Пользователь предоставляет вам два потока — один для чтения, другой для записи, — и вы копируете байты из одного в другой. В ходе этого процесса вполне возможно, что любой из потоков завершится неудачей, и в этот момент копирование должно остановиться и вернуть пользователю ошибку. Здесь пользователь, вероятно, захочет знать, произошёл ли сбой во входном или в выходном потоке. Например, в веб-сервере, если ошибка возникает во входном потоке при потоковой передаче файла клиенту, это может быть связано с тем, что диск извлечён, тогда как если ошибка возникает в выходном потоке, возможно, клиент просто отключился. Последнее может быть ошибкой, которую серверу следует проигнорировать, поскольку копирование в новые соединения по-прежнему может завершаться успешно, тогда как первое может потребовать отключения всего сервера!

Это случай, когда мы хотим перечислить ошибки. Пользователю необходимо иметь возможность различать разные случаи ошибок, чтобы реагировать соответствующим образом, поэтому мы используем `enum` с именем `CopyError`, где каждый вариант представляет отдельную первопричину ошибки, как в листинге 5-1.

```
pub enum CopyError {  
    In(std::io::Error),  
    Out(std::io::Error),  
}
```

Листинг 5-1: Перечислимый тип ошибки

Каждый вариант также включает встретившуюся ошибку, чтобы предоставить вызывающему коду как можно больше информации о том, что пошло не так.

Создавая собственный тип ошибки, вам нужно выполнить ряд шагов, чтобы этот тип ошибки хорошо взаимодействовал с остальной экосистемой Rust. Во-первых, ваш тип ошибки должен реализовывать трейт (trait) `std::error::Error`, который предоставляет вызывающим методам общие методы для

интроспекции типов ошибок. Главный интересующий нас метод — `Error::source`, который предоставляет механизм для нахождения первопричины ошибки. Чаще всего он используется для печати трассировки (`backtrace`), отображающей путь вплоть до корневой причины ошибки. Для нашего типа `CopyError` реализация `source` проста: мы сопоставляем `self` и извлекаем и возвращаем внутреннюю `std::io::Error`.

Во-вторых, ваш тип должен реализовывать как `Display`, так и `Debug`, чтобы вызывающий код мог осмысленно печатать вашу ошибку. Это обязательно, если вы реализуете трейт `Error`. В общем случае ваша реализация `Display` должна давать однострочное описание того, что пошло не так, которое можно легко встроить в другие сообщения об ошибках. Формат вывода должен быть в нижнем регистре и без завершающей пунктуации, чтобы он хорошо вписывался в другие, более крупные отчёты об ошибках. `Debug` должен предоставлять более описательную информацию об ошибке, включая вспомогательную информацию, которая может быть полезна при отслеживании причины ошибки, такую как номера портов, идентификаторы запросов, пути к файлам и тому подобное, для чего обычно достаточно `#[derive(Debug)]`.

ПРИМЕЧАНИЕ В более старом коде на Rust вы можете встретить ссылки на метод `Error::description`, но он был объявлен устаревшим в пользу `Display`.

В-третьих, ваш тип должен, по возможности, реализовывать как `Send`, так и `Sync`, чтобы пользователи могли совместно использовать ошибку через границы потоков. Если ваш тип ошибки не является потокобезопасным, вы обнаружите, что почти невозможно использовать ваш крейт в многопоточном контексте. Типы ошибок, реализующие `Send` и `Sync`, также гораздо проще использовать с очень распространённым типом `std::io::Error`, который способен оборачивать ошибки, реализующие `Error`, `Send` и `Sync`. Разумеется, не все типы ошибок разумно делать `Send` и `Sync` — например, если они привязаны к конкретным потоковым ресурсам, — и это нормально. Вы, вероятно, и не отправляете эти ошибки через границы потоков. Тем не менее, это нечто, о чём стоит помнить, прежде чем помещать типы `Rc<String>` и `RefCell<bool>` в свои ошибки.

Наконец, по возможности ваш тип ошибки должен быть `'static`. Самое непосредственное преимущество этого в том, что это позволяет вызывающему

коду легче распространять вашу ошибку вверх по стеку вызовов, не сталкиваясь с проблемами времён жизни (lifetime). Это также позволяет более легко использовать ваш тип ошибки с типами ошибок со стёртым типом (type-erased), как мы вскоре увидим.

Непрозрачные ошибки (Opaque Errors)

Теперь рассмотрим другой пример: библиотеку для декодирования изображений. Вы даёте библиотеке кучу байтов для декодирования, и она предоставляет вам доступ к различным методам манипуляции изображениями. Если декодирование завершается неудачей, пользователю нужно иметь возможность понять, как решить проблему, а значит, он должен понять причину. Но важно ли, заключается ли причина в недопустимом значении поля `size` в заголовке изображения или в сбое алгоритма сжатия при распаковке блока? Вероятно, нет — приложение не сможет осмысленно восстановиться ни в одной из этих ситуаций, даже если оно знает точную причину. В случаях подобных этому вы как автор библиотеки можете вместо этого предоставить единственный непрозрачный тип ошибки. Это также делает вашу библиотеку немного приятнее в использовании, поскольку повсюду используется только один тип ошибки. Этот тип ошибки должен реализовывать `Send`, `Debug`, `Display` и `Error` (включая метод `source`, где это уместно), но помимо этого вызывающему коду не нужно знать что-либо ещё. Вы можете внутренне представлять более детализированные состояния ошибок, но нет необходимости раскрывать их пользователям библиотеки. Это лишь без необходимости увеличило бы размер и сложность вашего API.

Как именно должен выглядеть ваш непрозрачный тип ошибки — в основном на ваше усмотрение. Это может быть просто тип со всеми приватными полями, который раскрывает лишь ограниченные методы для отображения и интроспекции ошибки, либо это может быть сильно стёртый по типу тип ошибки вроде `Box<dyn Error + Send + Sync + 'static>`, который не раскрывает ничего, кроме того факта, что это ошибка, и обычно вообще не позволяет вашим пользователям проводить интроспекцию. Решение о том, насколько непрозрачными делать ваши типы ошибок, в основном сводится к вопросу, есть ли в ошибке что-либо интересное помимо её описания. С `Box<dyn Error>` вы оставляете пользователям мало выбора, кроме как пробросить вашу ошибку наверх. Это может быть приемлемо, если у неё действительно нет ценной информации для представления пользователю — например, если это просто

динамическая ошибка или одна из большого числа несвязанных ошибок из более глубоких частей вашей программы. Но если у ошибки есть некоторые интересные грани, такие как номер строки или код состояния, вы можете захотеть раскрыть их через конкретный, но непрозрачный тип.

ПРИМЕЧАНИЕ В общем случае в сообществе сложился консенсус, что ошибки должны быть редкими и поэтому не должны существенно увеличивать стоимость «счастливого пути» (happy path). По этой причине ошибки часто помещают за тип-указатель, такой как `Box` или `Arc`. Таким образом они вряд ли существенно увеличат размер общего типа `Result`, в который они заключены.

Одно из преимуществ использования ошибок со стёртым типом в том, что они позволяют вам легко комбинировать ошибки из разных источников, не вводя дополнительных типов ошибок. То есть ошибки со стёртым типом часто хорошо *компонуются* (compose) и позволяют вам выражать открытый набор ошибок. Если вы пишете функцию, возвращаемый тип которой — `Box<dyn Error + ...>`, то вы можете использовать `?` для самых разных типов ошибок внутри этой функции, для всевозможных разных ошибок, и все они будут преобразованы в этот один общий тип ошибки.

`'static` в `Box<dyn Error + Send + Sync + 'static>` стоит того, чтобы потратить немного больше времени в контексте стирания. В предыдущем разделе я упомянул, что это полезно, чтобы позволить вызывающему коду распространять ошибку, не беспокоясь о границах времён жизни метода, но он служит и ещё большей цели: доступ к понижающему приведению (downcasting). *Понижающее приведение* (downcasting) — это процесс взятия элемента одного типа и приведения его к более конкретному типу. Это один из немногих случаев, когда Rust даёт вам доступ к информации о типах во время выполнения; это ограниченный случай более общей рефлексии типов, которую часто предоставляют динамические языки. В контексте ошибок понижающее приведение позволяет пользователю превратить `dyn Error` в конкретный нижележащий тип ошибки, когда `dyn Error` изначально был именно этим типом. Например, пользователь может захотеть предпринять определённое действие, если полученная им ошибка была `std::io::Error` вида `std::io::ErrorKind::WouldBlock`, но не предпринимать то же действие ни в каком другом случае. Если пользователь получает `dyn Error`, он может использовать `Error::downcast_ref`, чтобы попытаться выполнить понижающее приведение

ошибки к `std::io::Error`. Метод `downcast_ref` возвращает `Option`, который сообщает пользователю, удалось ли понижающее приведение. И здесь ключевое наблюдение: `downcast_ref` работает только в том случае, если аргумент является `'static`. Если мы вернём непрозрачную `Error`, которая не является `'static`, мы лишим пользователя возможности проводить такого рода интроспекцию ошибок, если он того пожелает.

В экосистеме есть некоторые разногласия по поводу того, являются ли ошибки со стёртым типом библиотеки (или, в более общем смысле, её типы со стёртым типом) частью её публичного и стабильного API. То есть, если метод `foo` в вашей библиотеке возвращает `lib::MyError` как `Box<dyn Error>`, будет ли изменение `foo` на возврат другого типа ошибки ломающим изменением (breaking change)? Сигнатура типа не изменилась, но пользователи могли написать код, который предполагает, что они могут выполнить понижающее приведение, чтобы превратить эту ошибку обратно в `lib::MyError`. Моё мнение по этому вопросу: вы выбрали возврат `Box<dyn Error>` (а не `lib::MyError`) по какой-то причине, и, если это явно не задокументировано, это не гарантирует ничего конкретного относительно понижающего приведения.

ПРИМЕЧАНИЕ Хотя `Box<dyn Error + ...>` — привлекательный тип ошибки со стёртым типом, он, как ни странно, сам *не* реализует `Error`. Поэтому подумайте о добавлении собственного типа `BoxError` для стирания типов в библиотеках, который *действительно* реализует `Error`.

Вы можете задаться вопросом, как `Error::downcast_ref` может быть безопасным. То есть, как он узнаёт, действительно ли предоставленный аргумент `dyn Error` имеет заданный тип `T`? В стандартной библиотеке даже есть трейт под названием `Any`, который реализуется для *любого* типа и который реализует `downcast_ref` для `dyn Any` — как это может быть нормально? Ответ кроется в поддерживаемом компилятором типе `std::any::TypeId`, который позволяет получить уникальный идентификатор для любого типа. У трейта `Error` есть скрытый предоставленный метод под названием `type_id`, реализация по умолчанию которого — возвращать `TypeId::of::<Self>()`. Аналогично, у `Any` есть бланкетная (blanket) реализация `impl Any for T`, и в этой реализации его `type_id` возвращает то же самое. В контексте этих блоков `impl` конкретный тип `Self` известен, поэтому этот `type_id` является идентификатором типа реального типа. Это предоставляет всю информацию, которая нужна `downcast_ref`. `downcast_ref` вызывает `self.type_id`, который проходит через виртуальную

таблицу (vtable) для типов динамического размера (см. главу 3) к реализации для нижележащего типа и сравнивает его с идентификатором типа предоставленного типа для понижающего приведения. Если они совпадают, то тип, скрытый за `dyn Error` или `dyn Any`, действительно является `T`, и безопасно приводить ссылку с одного к другому.

Особые случаи ошибок (Special Error Cases)

Некоторые функции являются сбоеопасными (fallible), но не могут вернуть никакой осмысленной ошибки при сбое. Концептуально такие функции имеют возвращаемый тип `Result<T, ()>`. В некоторых кодовых базах вы можете увидеть это представленным как `Option<T>`. Хотя оба варианта являются законным выбором возвращаемого типа для такой функции, они передают разные семантические значения, и обычно следует избегать «упрощения» `Result<T, ()>` до `Option<T>`. `Err(())` указывает на то, что операция завершилась неудачей и должна быть повторена, зарегистрирована или обработана в исключительном порядке. `None`, с другой стороны, передаёт лишь то, что функции нечего возвращать; обычно это не считается исключительным случаем или чем-то, что следует обрабатывать. Вы можете увидеть это в аннотации `#[must_use]` на типе `Result` — когда вы получаете `Result`, язык ожидает, что важно обработать оба случая, тогда как с `Option` ни один из случаев фактически не требует обработки.

ПРИМЕЧАНИЕ Вам также следует иметь в виду, что `()` не реализует трейт `Error`. Это означает, что её нельзя стереть по типу в `Box<dyn Error>` и она может быть немного мучительной в использовании с `?`. По этой причине в таких случаях часто лучше определить собственный тип-структуру без полей (unit struct), реализовать для него `Error` и использовать его в качестве ошибки вместо `()`.

Некоторые функции, например те, что запускают непрерывно работающий цикл сервера, возвращают ошибки только когда-либо; если только не возникает ошибка, они работают вечно. Другие функции никогда не дают ошибок, но тем не менее должны возвращать `Result`, например, чтобы соответствовать сигнатуре трейта. Для функций подобных этим Rust предоставляет тип *никогда* (never type), записываемый с помощью синтаксиса `!`. Тип «никогда» представляет значение, которое никогда не может быть сгенерировано. Вы не можете сконструировать экземпляр этого типа

самостоятельно — единственный способ создать его — войти в бесконечный цикл или вызвать панику (panicking), либо через горстку других специальных операций, о которых компилятор знает, что они никогда не возвращают управление. С `Result`, когда у вас есть `Ok` или `Err`, который, как вы знаете, никогда не будет использован, вы можете установить его в тип `!`. Если вы пишете функцию, которая возвращает `Result<T, !>`, вы не сможете когда-либо вернуть `Err`, поскольку единственный способ сделать это — войти в код, который никогда не вернёт управление. Поскольку компилятор знает, что любой вариант с `!` никогда не будет создан, он также может оптимизировать ваш код с учётом этого, например, не генерируя код паники для `unwrap` на `Result<T, !>`. А когда вы выполняете сопоставление с образцом (pattern match), компилятор знает, что любой вариант, содержащий `!`, даже не нужно перечислять. Довольно изящно!

Последний любопытный случай ошибки — тип ошибки `std::thread::Result`. Вот его определение:

```
type Result<T> = Result<T, Box<dyn Any + Send + 'static>>;
```

Тип ошибки имеет стёртый тип, но он не стёрт до `dyn Error`, как мы видели до сих пор. Вместо этого это `dyn Any`, который гарантирует лишь то, что ошибка имеет *некоторый* тип, и ничего больше... что является не такой уж и большой гарантией. Причина этого странно выглядящего типа ошибки в том, что вариант ошибки `std::thread::Result` создаётся только в ответ на панику; в частности, если вы попытаетесь присоединиться (`join`) к потоку, который запаниковал. В этом случае неясно, что присоединяющийся поток может сделать что-либо иное, кроме как либо проигнорировать ошибку, либо запаниковать самому с помощью `unwrap`. По сути, тип ошибки — это «паника», а значение — это «какой бы аргумент ни был передан в `panic!`», который действительно может быть любым типом (хотя обычно это форматированная строка).

Распространение ошибок (Propagating Errors)

Оператор `?` в Rust действует как сокращение для *развернуть или вернуться раньше времени* (`unwrap` or `return early`), для удобной работы с ошибками. Но в его рукаве есть и несколько трюков, о которых стоит знать. Во-первых, `?` выполняет преобразование типа через трейт `From`. В функции, которая

возвращает `Result<T, E>`, вы можете использовать `?` для любого `Result<T, X>`, где `E: From<X>`. Именно эта возможность делает стирание ошибок через `Box<dyn Error>` столь привлекательным; вы можете просто использовать `?` повсюду и не беспокоиться о конкретном типе ошибки, и обычно это просто «заработает».

FROM И INTO (FROM AND INTO) В стандартной библиотеке много трейтов преобразования, но два из основных — это `From` и `Into`. Вам может показаться странным иметь два: если у нас есть `From`, зачем нам `Into`, и наоборот? Тому есть несколько причин, но давайте начнём с исторической: в ранние дни Rust из-за правил когерентности (coherence rules), обсуждавшихся в главе 3, было бы невозможно иметь только один из них. Или, точнее, из-за того, какими *были* правила когерентности.

Предположим, вы хотите реализовать двустороннее преобразование между неким локальным типом, который вы определили в своём крейте, и неким типом из стандартной библиотеки. Вы можете без труда реализовать `impl<T> From<Vec<T>> for MyType<T>` и `impl<T> Into<Vec<T>> for MyType<T>`, но если бы у вас был только `From` или `Into`, вам пришлось бы реализовать `impl<T> From<MyType<T>> for Vec<T>` или `impl<T> Into<MyType<T>> for Vec<T>`. Однако компилятор раньше отклонял эти реализации! Только начиная с Rust 1.41.0, когда исключение для покрытых типов (covered types) было добавлено в правила когерентности, они стали легальны. До этого изменения было необходимо иметь оба трейта. И поскольку много кода на Rust было написано до Rust 1.41.0, ни один из трейтов нельзя удалить сейчас.

Однако, помимо этого исторического факта, есть и хорошие эргономические причины иметь оба этих трейта, даже если бы мы могли начать с нуля сегодня. Часто значительно проще использовать один или другой в разных ситуациях. Например, если вы пишете метод, который принимает тип, который можно превратить в `Foo`, что бы вы предпочли написать: `fn(impl Into<Foo>)` или `fn<T>(T) where Foo: From<T>?` И наоборот, чтобы превратить строку в синтаксический идентификатор, что бы вы предпочли написать: `Ident::from("foo")` или `<_ as Into<Ident>>::into("foo")`? У обоих этих трейтов есть свои применения, и нам лучше иметь оба.

Учитывая, что у нас есть оба, вы можете задаться вопросом, какой из

них использовать в собственном коде. Ответ, как выясняется, довольно прост: реализуйте `From` и используйте `Into` в ограничениях (`bounds`). Причина в том, что у `Into` есть бланкетная реализация для любого `T`, реализующего `From`, так что независимо от того, реализует ли тип явно `From` или `Into`, он реализует `Into`!

Разумеется, как это часто бывает с простыми вещами, на этом история не вполне заканчивается. Поскольку компилятору часто приходится «проходить через» бланкетную реализацию, когда `Into` используется в качестве ограничения, рассуждение о том, реализует ли тип `Into`, сложнее, чем о том, реализует ли он `From`. И в некоторых случаях компилятор недостаточно умен, чтобы разгадать эту головоломку. По этой причине оператор `?` на момент написания книги использует `From`, а не `Into`. В большинстве случаев это не имеет значения, поскольку большинство типов реализуют `From`, но это означает, что типы ошибок из старых библиотек, которые вместо этого реализуют `Into`, могут не работать с `?`. По мере того как компилятор становится умнее, `?`, вероятно, будет «обновлён» до использования `Into`, и в этот момент эта проблема исчезнет, но пока что у нас есть то, что есть.

Второй аспект `?`, о котором стоит знать, — что этот оператор на самом деле является лишь синтаксическим сахаром для трейта, предварительно названного `Try`. На момент написания книги трейт `Try` ещё не был стабилизирован, но к тому времени, когда вы будете читать это, он или нечто очень похожее, вероятно, устоится. Поскольку детали ещё не все определены, я дам вам лишь общий очерк того, как работает `Try`, а не полные сигнатуры методов. В своей основе `Try` определяет тип-обёртку, состояние которого либо такое, в котором дальнейшие вычисления полезны (счастливый путь), либо такое, в котором они бесполезны. Некоторые из вас правильно подумают о монадах, хотя мы не будем здесь исследовать эту связь. Например, в случае `Result<T, E>`, если у вас есть `Ok(t)`, вы можете продолжить по счастливому пути, развернув `t`. Если же у вас есть `Err(e)`, с другой стороны, вы хотите прекратить выполнение и немедленно произвести значение ошибки, поскольку дальнейшие вычисления невозможны, так как у вас нет `t`.

Что интересно в `Try`, так это то, что он применим к большему числу типов, чем просто `Result`. `Option<T>`, например, следует тому же паттерну — если у вас есть `Some(t)`, вы можете продолжить по счастливому пути, тогда как если у вас есть

None, вы хотите выдать None вместо продолжения. Этот паттерн распространяется на более сложные типы, такие как `Poll<Result<T, E>>`, тип счастливого пути которого — `Poll<T>`, что делает `?` применимым в гораздо большем числе случаев, чем вы могли бы ожидать. Когда `Try` стабилизируется, мы, возможно, увидим, что `?` начнёт работать со всевозможными типами, чтобы сделать наш код счастливого пути приятнее.

Оператор `?` уже можно использовать в сбоеопасных функциях, в `doctest`-ах и в `fn main`. Однако чтобы раскрыть его полный потенциал, нам также нужен способ ограничить область видимости (`scope`) этой обработки ошибок. Например, рассмотрим функцию в листинге 5-2.

```
fn do_the_thing() -> Result<(), Error> {
    let thing = Thing::setup()?;
    // .. код, использующий thing, и ? ..
    thing.cleanup();
    Ok(())
}
```

Листинг 5-2: Многошаговая сбоеопасная функция, использующая оператор `?`

Это не вполне сработает так, как ожидается. Любой `?` между `setup` и `cleanup` вызовет ранний возврат из всей функции, что пропустит код очистки! Это проблема, которую призваны решить *try-блоки* (`try blocks`). `try`-блок ведёт себя во многом как цикл с одной итерацией, где `?` использует `break` вместо `return`, а финальное выражение блока имеет неявный `break`. Теперь мы можем исправить код из листинга 5-2 так, чтобы он всегда выполнял очистку, как показано в листинге 5-3.

```
fn do_the_thing() -> Result<(), Error> {
    let thing = Thing::setup()?;
    let r = try {
        // .. код, использующий thing, и ? ..
    };
    thing.cleanup();
    r
}
```

Листинг 5-3: Многошаговая сбоеопасная функция, которая всегда выполняет очистку после себя

`try`-блоки на момент написания книги также не стабилизированы, но есть достаточный консенсус относительно их полезности, так что они, вероятно,

появятся в форме, похожей на описанную здесь.

Итоги (Summary)

Эта глава охватила два основных способа конструирования типов ошибок в Rust: перечисление и стирание. Мы рассмотрели, когда вы можете захотеть использовать каждый из них, а также преимущества и недостатки каждого. Мы также взглянули на некоторые закулисные аспекты оператора `?` и подумали о том, как `?` может стать ещё полезнее в будущем. В следующей главе мы сделаем шаг назад от кода и посмотрим, как вы *структурируете* проект на Rust. Мы рассмотрим флаги функций (feature flags), управление зависимостями и версионирование, а также то, как управлять более сложными крейтами с помощью рабочих пространств (workspaces) и подкрейтов (subcrates). Увидимся на следующей странице!